ONYX Archiver

System für sicheren Datenaustausch und kryptografische Archivierung.

1. Projektziel

Angesichts moderner Cybersicherheitsbedrohungen bieten Standard-Archivierungswerkzeuge keinen ausreichenden Schutz bei der Übertragung von Dateien.

Ziel: Entwicklung einer Desktop-Anwendung zur Erstellung geschützter Archive, auf die ausschließlich autorisierte Empfänger zugreifen können, die zuvor ein Schlüsselaustauschverfahren durchlaufen haben.

2. Hauptfunktionen

- **Proprietäres .onx-Format:** Hybride Verschlüsselung von Dateien und Ordnern. Die Daten sind durch Algorithmen geschützt, die resistent gegen unbefugtes Abfangen sind.
- **Peer-to-Peer Handshake:** Ein einzigartiges System zum Austausch öffentlicher Schlüssel über spezielle Handshake-Dateien (`.onxk`). Dies gewährleistet eine konsequente End-to-End-Verschlüsselung.
- **Sicheres Benutzerprofil:** Speicherung von Daten in einer verschlüsselten lokalen SQLite-Datenbank unter Verwendung von **Argon2id**-Passworthashing, **X25519**-Verschlüsselungsschlüsseln und **XChaCha20-Poly1305**-Authentifizierter Verschlüsselung.
- **Kontaktverwaltung:** Ein umfassendes System zur Verwaltung von „Peers“ (Kontakten) – einschließlich Hinzufügen, Überprüfung des Vertrauensstatus und Löschen.

3. Technologiestack

Für die Umsetzung wurden moderne Industriestandards gewählt:

- **Sprache:** C# 12 / .NET 8.
- **Architektur: MVVM (Model-View-ViewModel)** – zur strikten Schichtentrennung zwischen Geschäftslogik und Benutzeroberfläche.
- **UI: WPF** mit asynchroner Verarbeitung (das UI bleibt auch bei der Verschlüsselung großer Dateien stets reaktionsfähig).
- **DI (Dependency Injection):** Einsatz eines Inversion-of-Control-Containers zur Steigerung der Flexibilität und Testbarkeit des Codes.
- **Datenkompression: ZStandard** (für eine effiziente und schnelle Komprimierung).
- **Datenbank: SQLite** in Verbindung mit **Entity Framework Core**.
- **Verschlüsselung: XChaCha20-Poly1305** (Authentifizierte Verschlüsselung mit hoher Performance).
- **Schlüsselaustausch: X25519** in Kombination mit **HKDF** (Key Derivation Function).
- **Digitale Signaturen: Ed25519** (zur Sicherstellung der Datenintegrität und Authentizität).
- **Passwortschutz & Datenbank-Sicherheit: Argon2id** (als hochsicherer Key-Derivation-Algorithmus).

4. Besondere Merkmale (Unique Features)

- **Sicherheit an erster Stelle (Security First):** Die Anwendung implementiert das **„Zero-Trust-Prinzip“**. Alle Daten werden direkt auf der Seite des Absenders verschlüsselt und können ausschließlich vom autorisierten Empfänger entschlüsselt werden (**Ende-zu-Ende-Verschlüsselung**).
- **Asynchronität:** Sämtliche rechenintensiven Operationen (wie Verschlüsselungsprozesse und Datenbankzugriffe) werden in **Hintergrund-Threads** ausgelagert. Dies ermöglicht eine flüssige Bedienung der Benutzeroberfläche bei gleichzeitiger Anzeige des Fortschritts in Echtzeit.
- **Skalierbarkeit:** Durch ein **schnittstellenorientiertes Design** (Interface-oriented Design) ist das System hochflexibel. Neue Verschlüsselungsalgorithmen oder Cloud-Speicheranbindungen können problemlos integriert werden, ohne die bestehende Kernlogik zu beeinträchtigen.

5. Schlüsselmanagement-Architektur und Datenschutz

5.1. Verwaltung des Master-Keys und der Sitzungen

Das Fundament der Sicherheit bildet ein statischer privater X25519-Schlüssel, der bei der Benutzerregistrierung generiert wird. Dessen Schutz und Lebenszyklus sind wie folgt organisiert:

- **Ableitung und Speicherung (Derivation & Storage)**: Zum Schutz des Schlüssels wird das Benutzerpasswort verwendet. Mithilfe des Argon2id-Algorithmus, der HKDF-Funktion und eines einzigartigen Salts wird ein Key Encryption Key (KEK) erzeugt. Mit diesem KEK wird der private X25519-Schlüssel verschlüsselt, bevor er in der Datenbank gespeichert wird. Zusammen mit den Benutzerdaten wird ein kryptografischer Deskriptor (Salt- und Argon2id-Parameter) gespeichert, der für die Wiederherstellung des Zugriffs erforderlich ist.
- **Aktive Sitzung und DPAPI**: Beim System-Login wird der Master-Key basierend auf dem Deskriptor aus dem Passwort wiederhergestellt. Um eine Kompromittierung des Schlüssels durch Memory Dumps oder externe Prozesse zu verhindern, wird dieser im geschützten Windows DPAPI-Speicher (Data Protection API) abgelegt. Dies gewährleistet eine Bindung des Geheimnisses an die aktuelle Betriebssystem-Sitzung.
- **Sitzungsbeendigung**: Beim Betätigen der Schaltfläche „Logout“ wird der Master-Key umgehend aus der DPAPI gelöscht und alle temporären Daten im Arbeitsspeicher werden bereinigt. Dies schließt jeglichen Zugriff auf die verschlüsselte Datenbank bis zur nächsten legitimen Autorisierung aus.

5.2. Hybrides Verschlüsselungsschema der Datenbank

Für den unmittelbaren Schutz der Datensätze in der Datenbank wurde ein Schema auf Basis des **Diffie-Hellman-Protokolls über elliptische Kurven (X25519)** implementiert, das nach dem Prinzip von Einmalschlüsseln (Session Keys) arbeitet:

- **Generierung des Operationsschlüssels:** Für jeden einzelnen Schreibvorgang wird ein **ephemeres (temporäres) Schlüsselpaar** generiert. Auf Basis des statischen öffentlichen Schlüssels des Benutzers und des privaten Schlüssels des ephemeren Paares wird ein gemeinsames Geheimnis (**Shared Secret**) mittels des **ECDH-Algorithmus** berechnet.

- **Ableitung des Sitzungsschlüssels (Key Derivation):** Das resultierende Geheimnis wird über die **HKDF-Funktion** unter Verwendung eines einzigartigen Salts in den finalen symmetrischen Verschlüsselungsschlüssel umgewandelt. Dies garantiert, dass die Chiffretexte in der Datenbank selbst bei identischen Eingabedaten immer einzigartig sind.
- **Metadaten zur Entschlüsselung:** Um das spätere Lesen zu ermöglichen, werden im Header jedes verschlüsselten Datensatzes die notwendigen öffentlichen Komponenten gespeichert: das **einzigartige Salt** und der **öffentliche Schlüssel des ephemeren Paares**. Dies erlaubt es dem rechtmäßigen Benutzer, das gemeinsame Geheimnis wiederherzustellen und die Daten zu entschlüsseln, ohne den Master-Key direkt offenlegen zu müssen.

6. Sicherer Datenaustausch und Handshake-Protokoll

6.1. Schlüsselaustausch-Protokoll (Handshake)

Vor Beginn des Datenaustauschs müssen die Benutzer ein „Handshake"-Verfahren durchführen, um ihre langfristigen öffentlichen Schlüssel auszutauschen.

- **Übertragungsmechanismus:** Der Austausch erfolgt über Dateien im proprietären Format `.onxk`. Die Struktur des Dateinamens (`handshake-[UUID]-[type].onxk`) ermöglicht es der Anwendung, die Sitzung sowie die jeweilige Phase des Prozesses (Initiierung/Bestätigung) eindeutig zu identifizieren.
- **Ergebnis:** Während des Handshakes tauschen die Parteien ihre öffentlichen Schlüssel für **X25519** (Verschlüsselung) und **Ed25519** (Authentizitätsprüfung/Signatur) aus. Dies schafft das Fundament für eine sichere Interaktion, ohne dass zentrale Server erforderlich sind.

6.2. Kryptografischer Schutz des Archivs

Der Prozess der Archiverstellung basiert auf dem Schema der **authentifizierten Verschlüsselung (Authenticated Encryption)** sowie Mechanismen zur Verschleierung von Metadaten:

- **Bildung des Sitzungsschlüssels:** Für jedes Archiv wird ein **ephemeres X25519-Paar** generiert. Auf Basis seiner privaten Komponente und des öffentlichen Schlüssels des Empfängers wird ein gemeinsames Geheimnis (**Shared Secret**) berechnet. Mithilfe der **HKDF-Funktion**

und eines einzigartigen Salts wird dieses Geheimnis in einen symmetrischen Verschlüsselungsschlüssel umgewandelt.
- **Integrität und Urheberschaft:** Das gesamte Datenpaket wird mit dem privaten **Ed25519-Schlüssel** des Absenders signiert. Dies garantiert den Schutz vor Datenmanipulation durch Dritte (Integritätsschutz).
- **Metadaten zur Entschlüsselung:** Dem Archiv werden der öffentliche Schlüssel des ephemeren Paares sowie das verwendete Salt beigefügt.

6.3. Schutz vor Traffic-Analyse und Identifizierung

Ein besonderes Merkmal des Systems ist die Art und Weise, wie der Empfänger den passenden Schlüssel identifiziert:

- **Dynamischer Identifikator (Key ID):** Anstatt eine explizite Benutzer-ID zu übertragen, berechnet der Absender einen **SHA-512-Hash** aus dem gemeinsamen Geheimnis (Shared Secret) und dem Salt. Da sowohl das Geheimnis als auch das Salt für jedes Archiv einzigartig sind, ist die Key ID bei jeder Übertragung unterschiedlich – selbst wenn das Archiv wiederholt an denselben Empfänger gesendet wird. Dies verhindert, dass Dritte durch die Analyse von Metadaten Rückschlüsse auf die Kommunikationsbeziehungen zwischen Benutzern ziehen können.
- **Entschlüsselungsprozess:** Der Empfänger identifiziert den Absender mittels **kryptografischen Probierens (Trial Decryption)**. Er generiert nacheinander gemeinsame Geheimnisse unter Verwendung der öffentlichen Schlüssel seiner bekannten Kontakte aus der lokalen Datenbank und vergleicht den resultierenden Hash mit der Key ID im Archiv. Bei einer Übereinstimmung gilt der Schlüssel als identifiziert und der eigentliche Entschlüsselungsprozess beginnt.

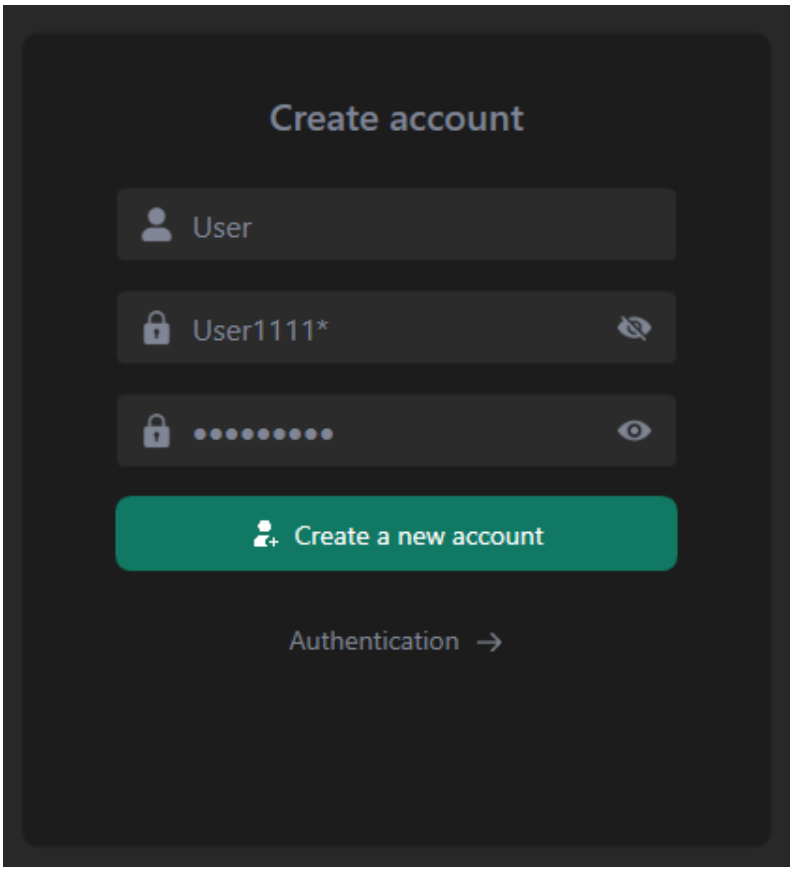
Fazit

ONYX Archiver ist nicht nur ein einfaches Hilfsprogramm, sondern ein fertiges Fundament für den sicheren geschäftlichen oder privaten Dokumentenaustausch. Das Projekt demonstriert die Anwendung komplexer kryptografischer Muster in Kombination mit einer benutzerfreundlichen, modernen Oberfläche.

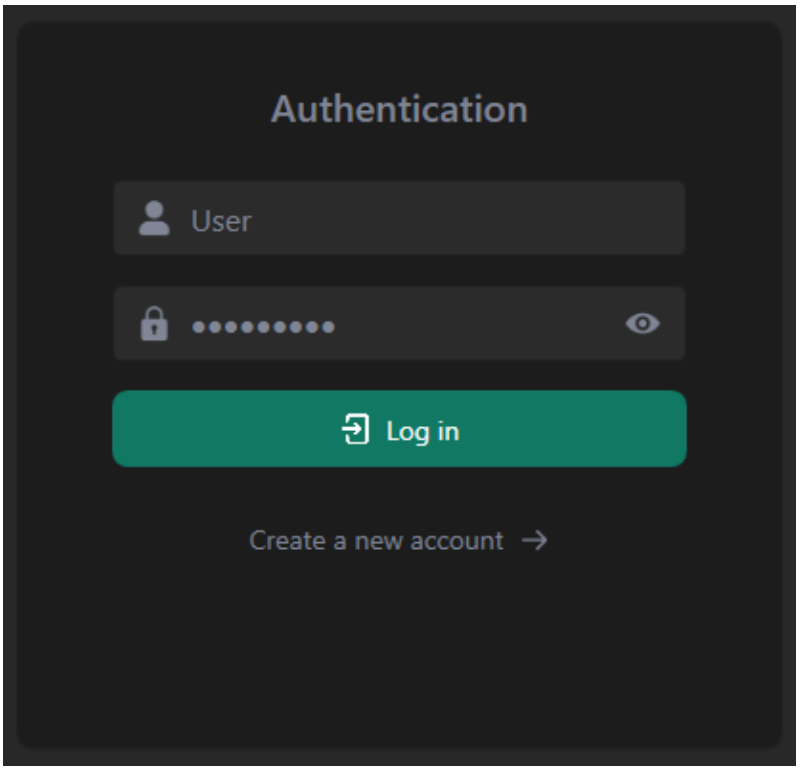
7. Funktionsübersicht der Software

7.1. Erstellung und Anmeldung am Benutzerkonto

7.1.1. Registrierungsformular (Konto erstellen)



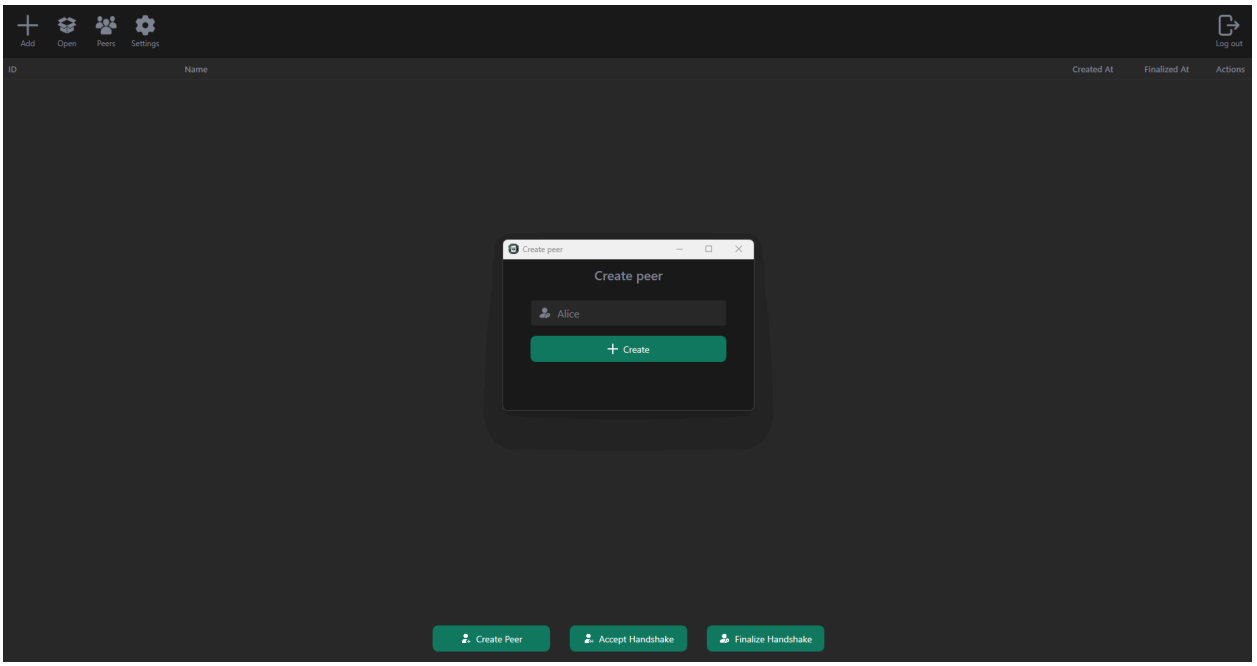
7.1.2. Anmeldemaske (Login-Formular)



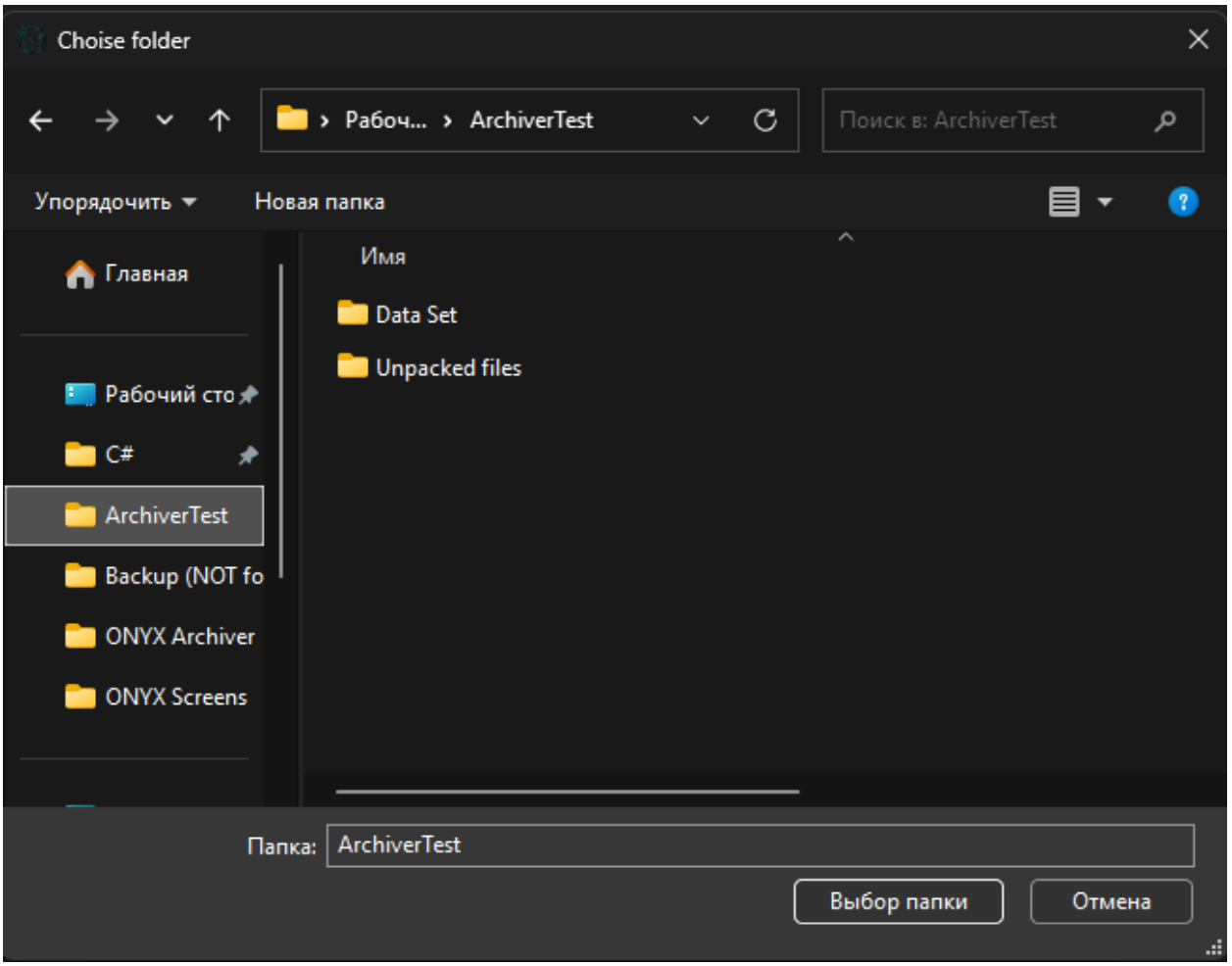
7.2. Peers (Knotenverwaltung)

7.2.1. Erstellung eines neuen Peers

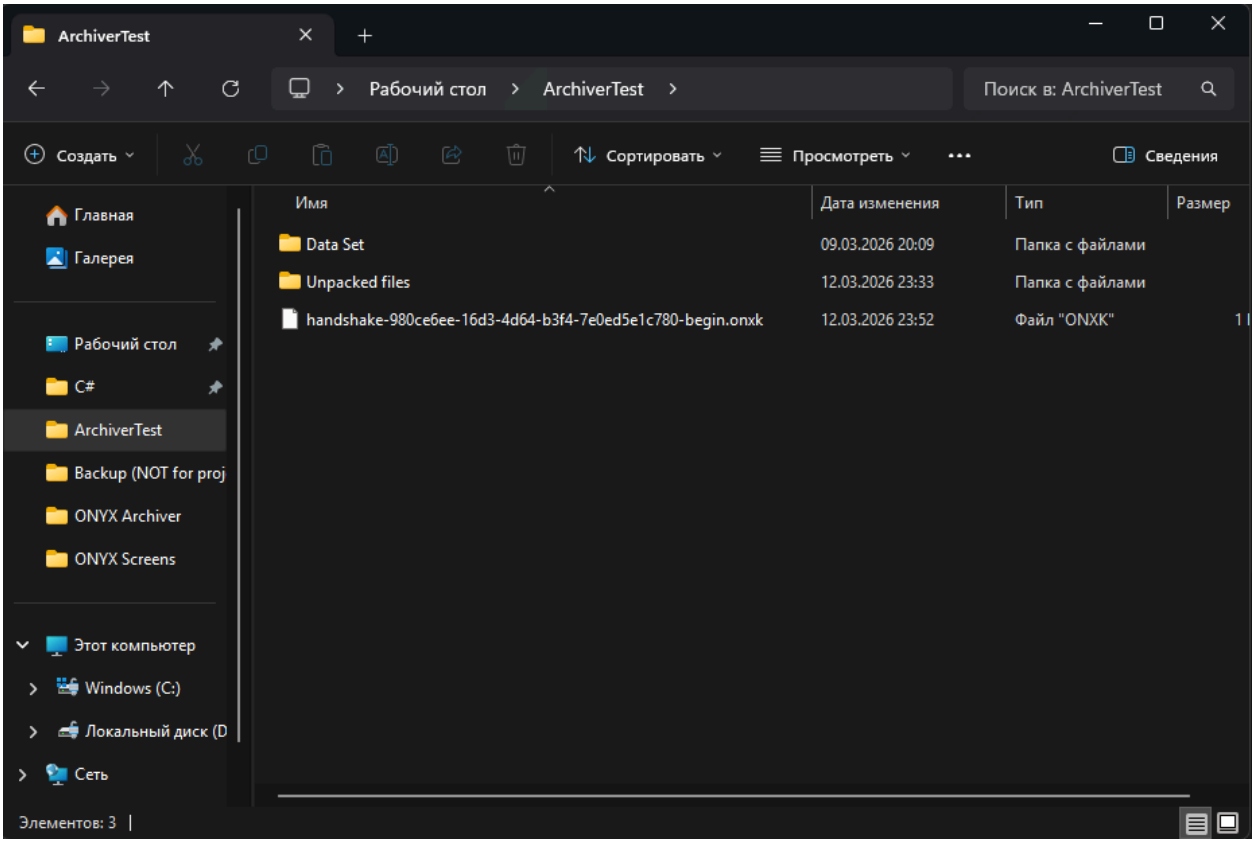
7.2.1.1. Eingabe des Peernamens



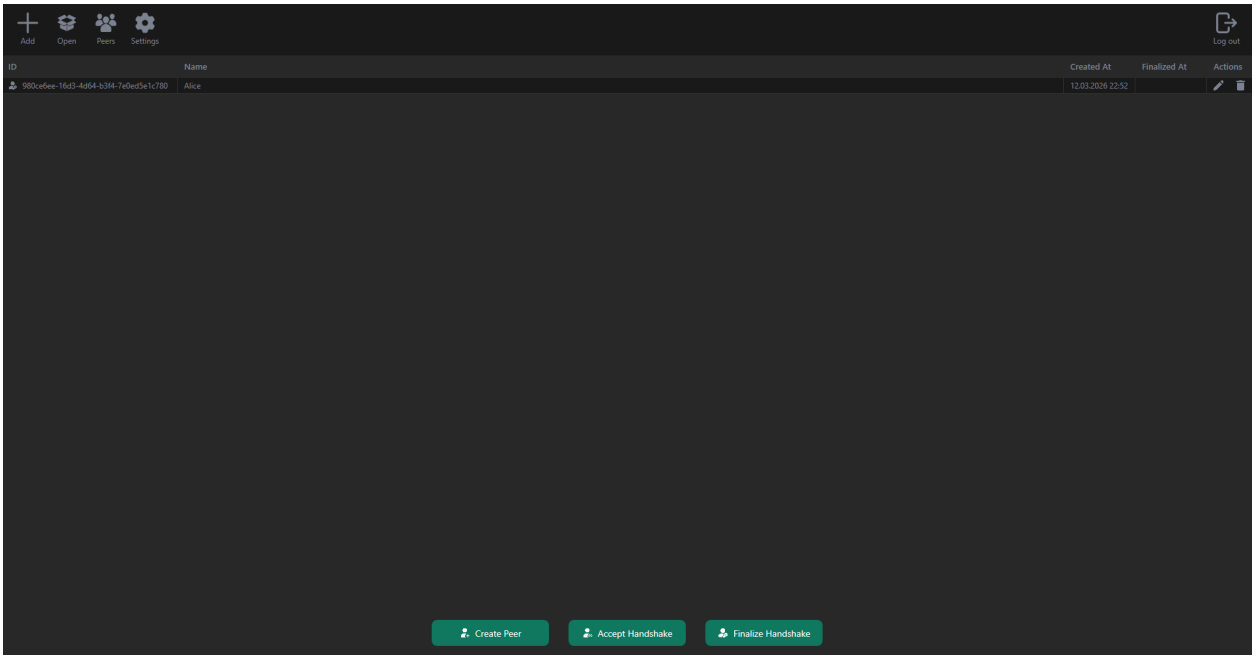
7.2.1.2. Zielverzeichnis für die Handshake-Datei festlegen



7.2.1.3. Handshake-Datei erfolgreich erstellt

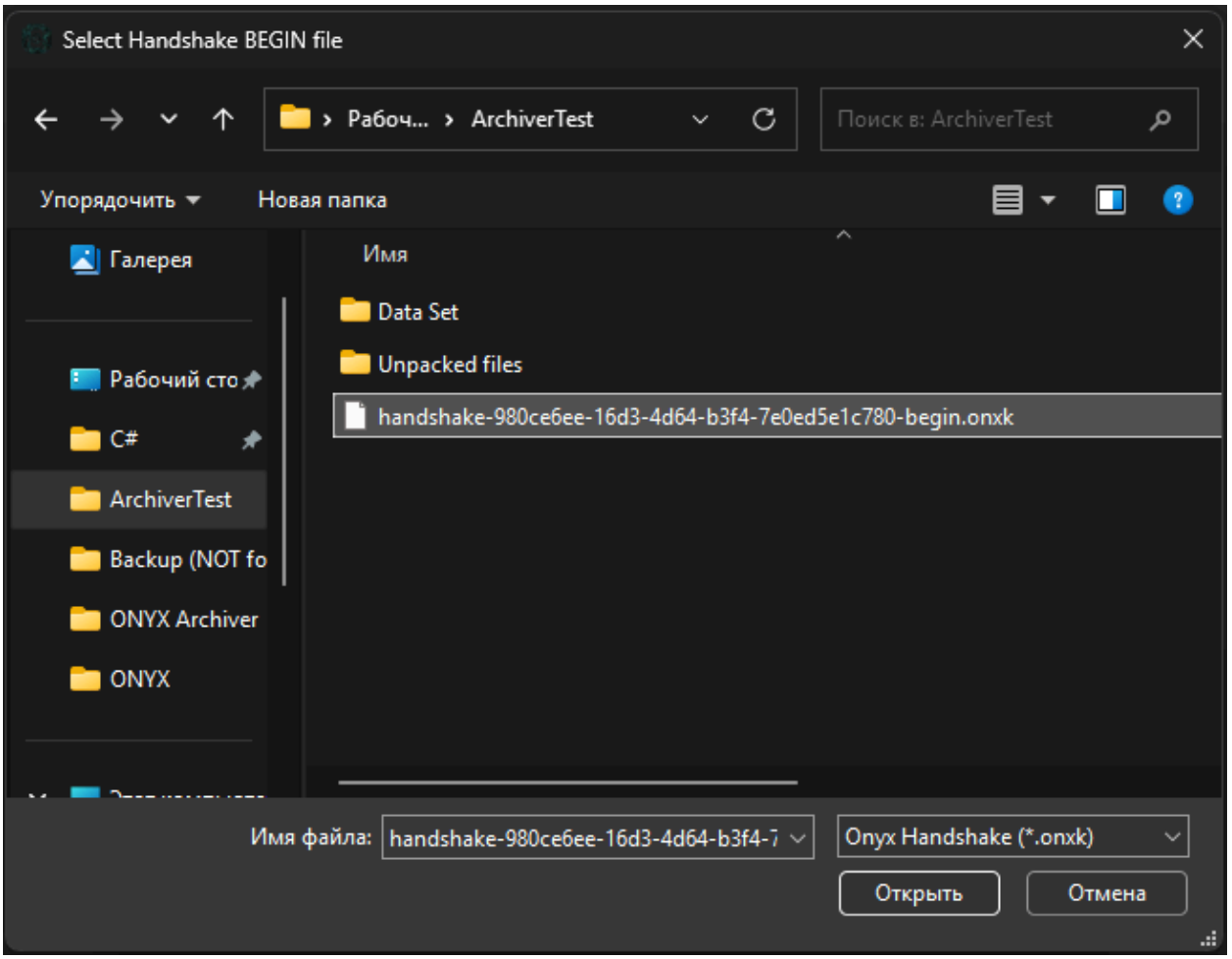


7.2.1.4. Speicherung des neuen Peer-Eintrags in der Datenbank

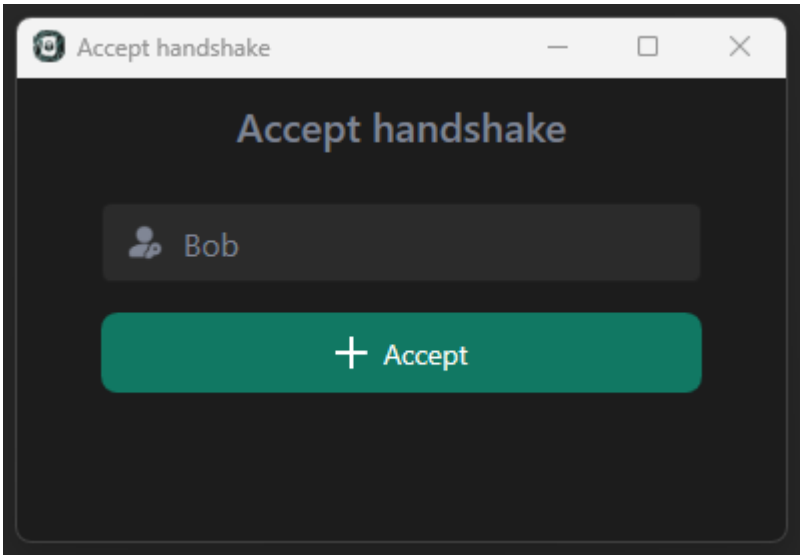


7.2.2. Bestätigung des Handshakes (Abschluss des Schlüsselaustauschs)

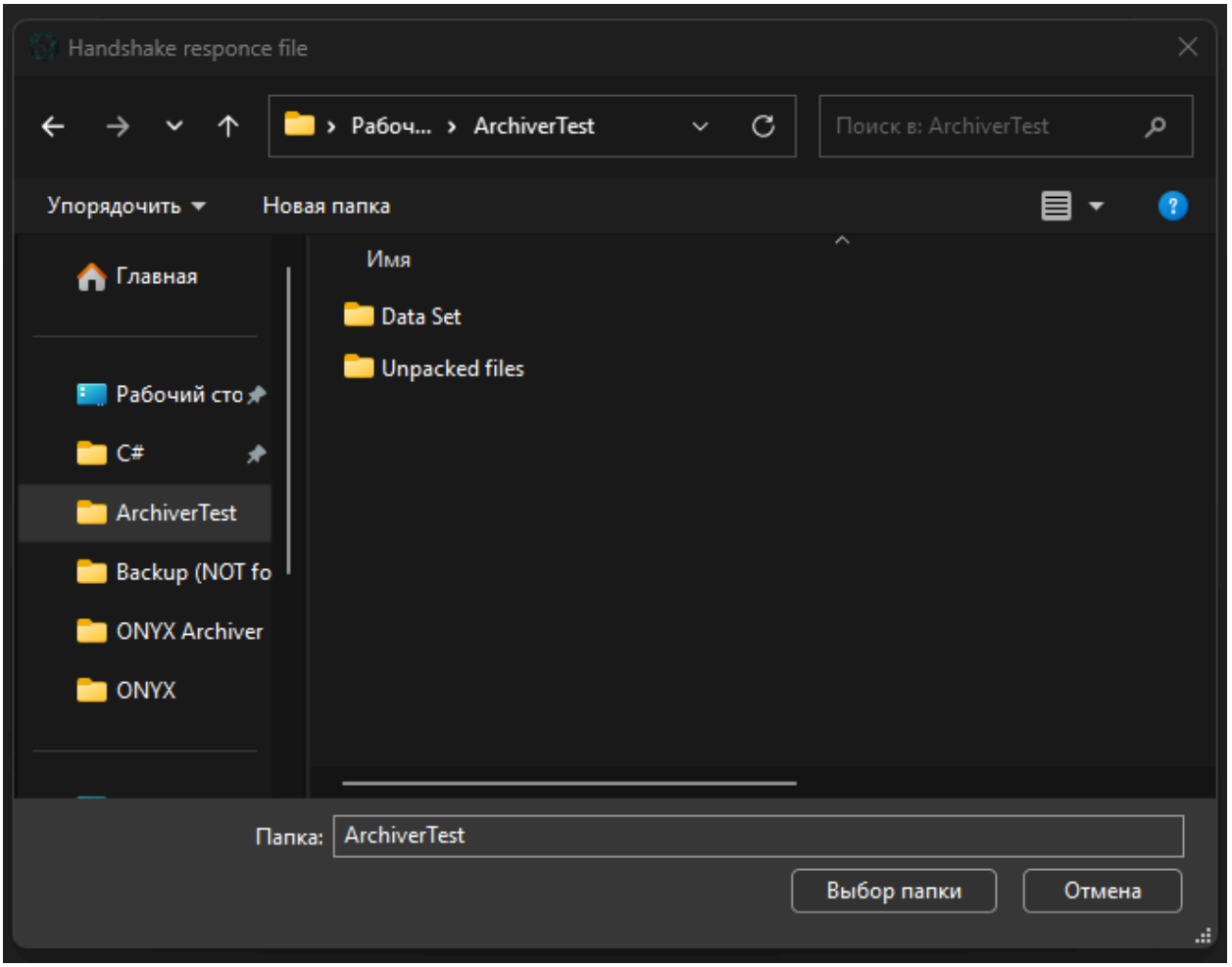
7.2.2.1. Auswahl der Handshake-Datei (Endung „-begin.onxk“)



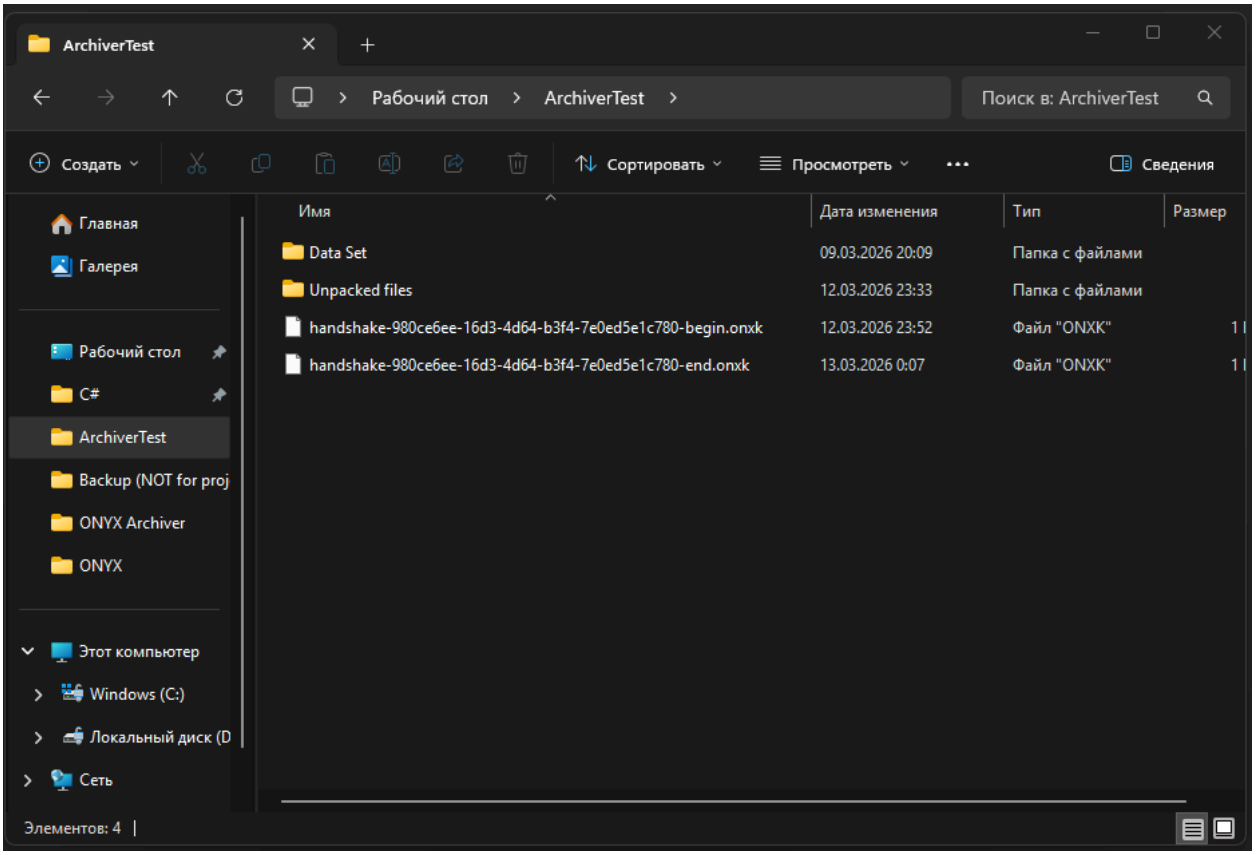
7.2.2.2. Eingabe des Namens für den neuen Peers



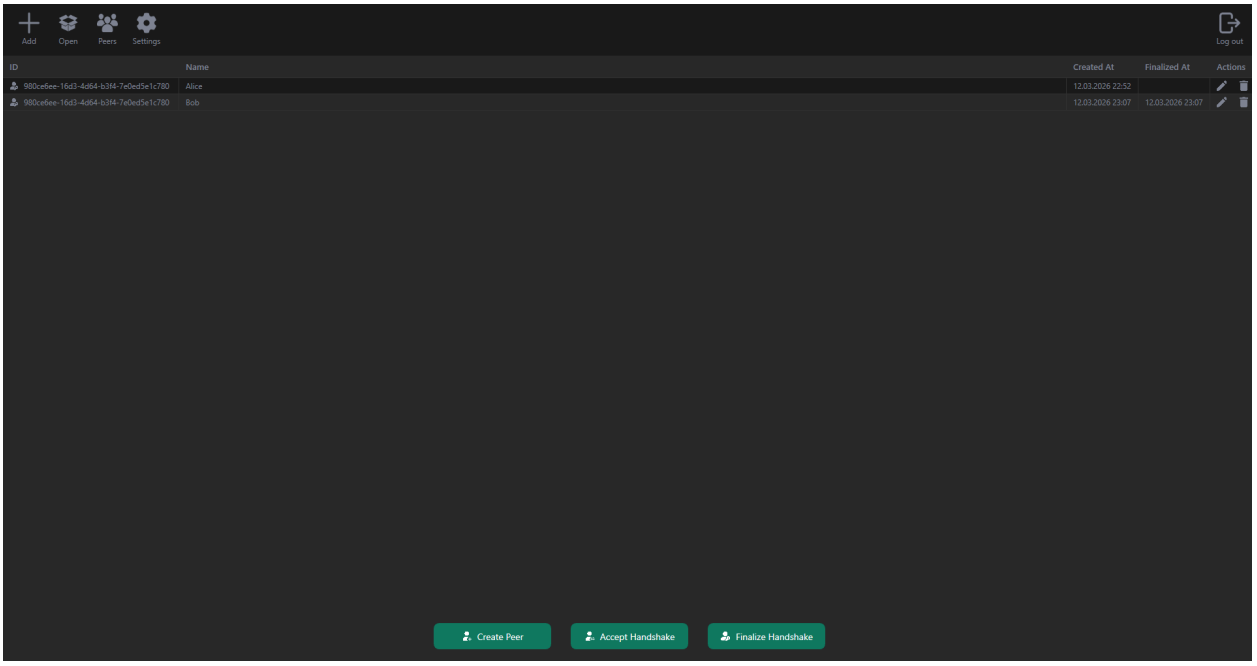
7.2.2.3. Zielverzeichnis für die Handshake-Antwortdatei festlegen



7.2.2.4. Handshake-Antwortdatei erfolgreich erstellt

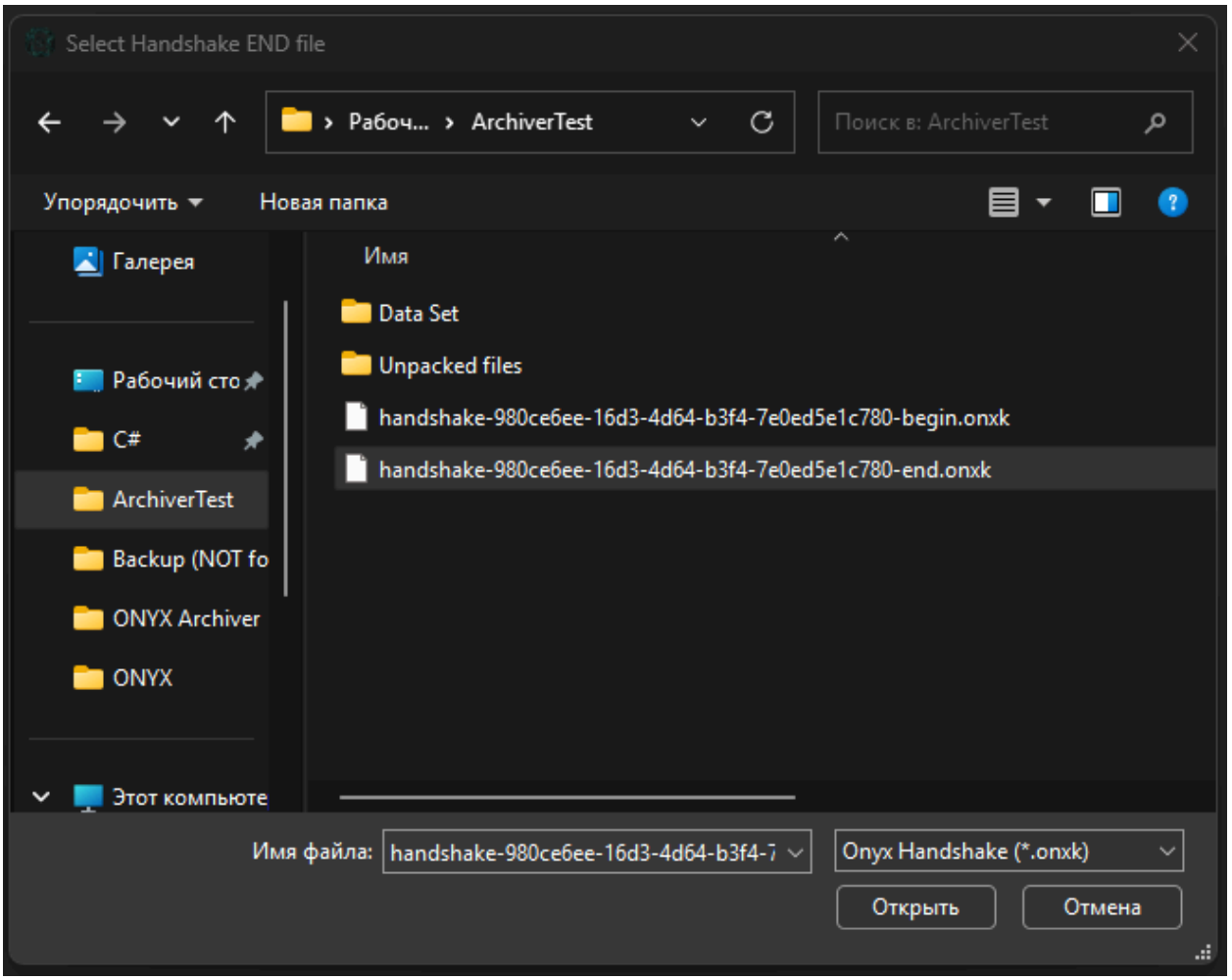


7.2.2.5. Registrierung des Partners in der Datenbank abgeschlossen

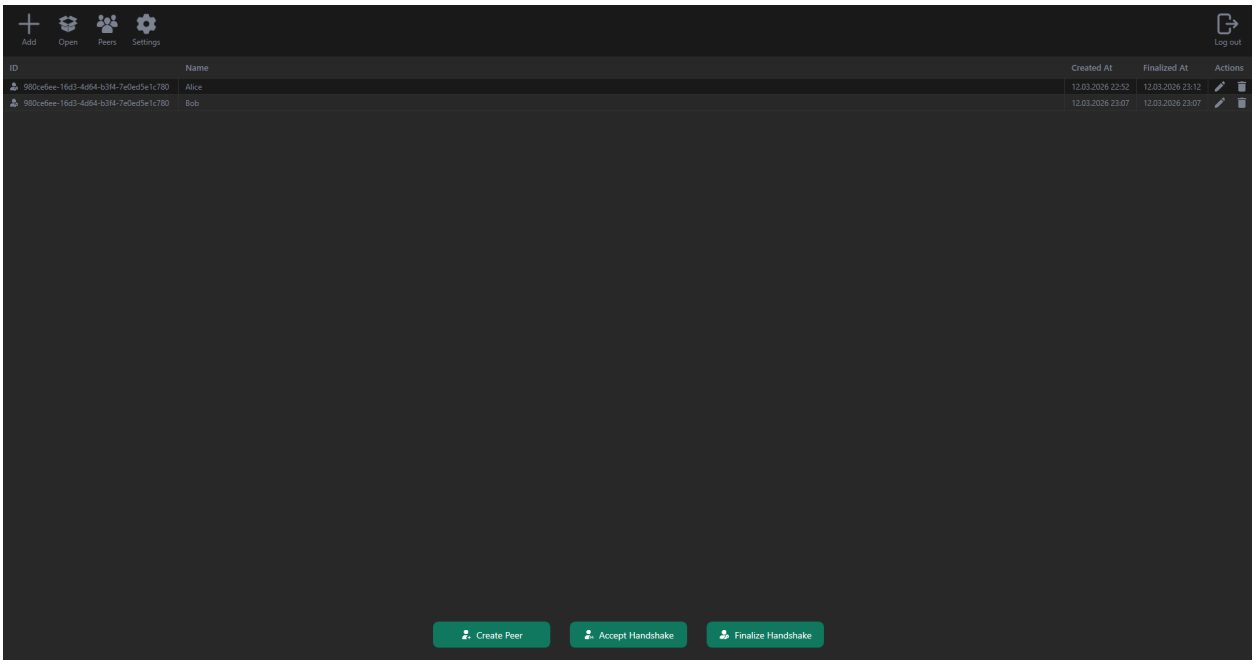


7.2.3. Abschluss des Handshakes

7.2.3.1. Auswahl der Handshake-Datei (Endung „-end.onxk“)

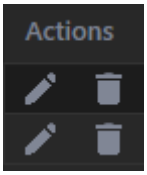


7.2.3.2. Handshake abgeschlossen und Aktualisierung des Feldes „Finalized At“

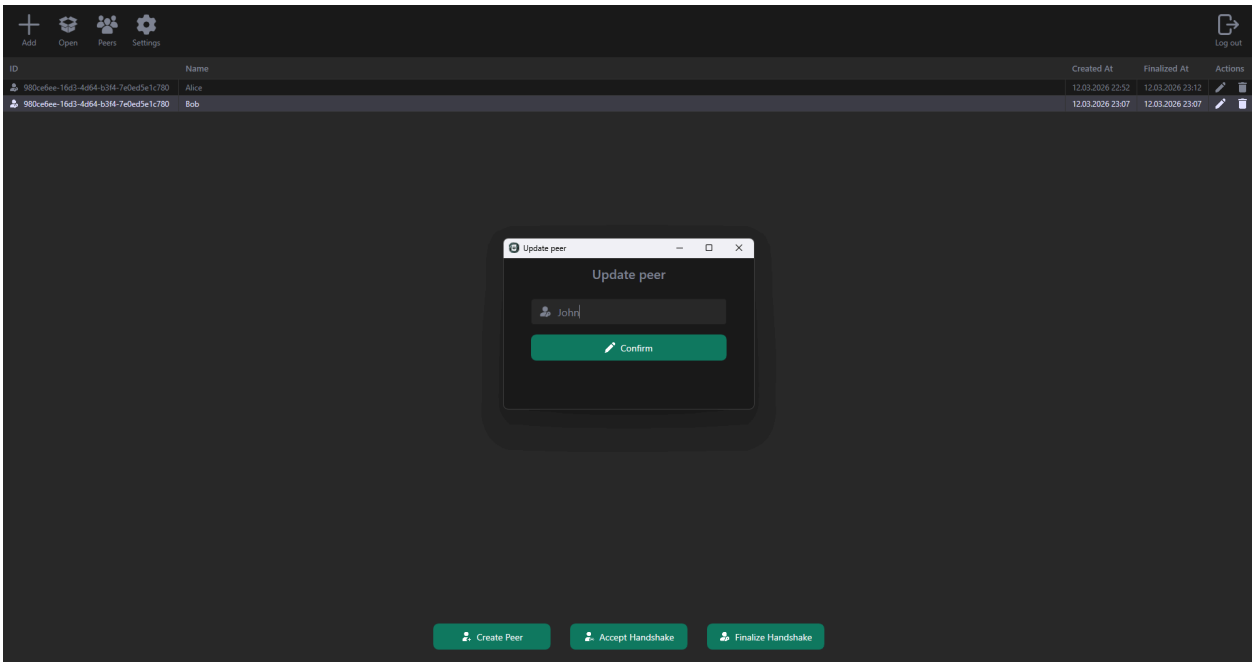


7.2.4. Umbenennung eines Peers

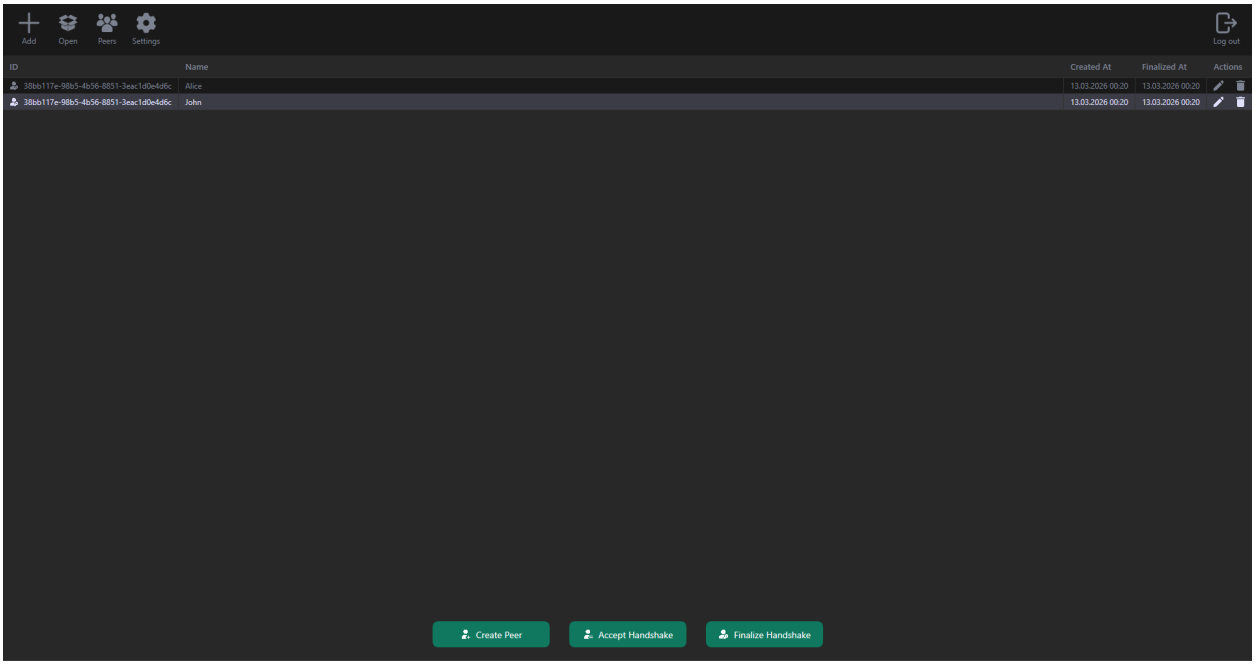
7.2.4.1. Klicken auf die Schaltfläche „Edit“ (links)



7.2.4.2. Eingabe des neuen Peer Namens

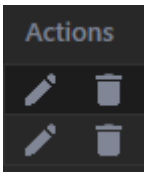


7.2.4.3. Änderung des Knotennamens von „Bob“ zu „John“ abgeschlossen

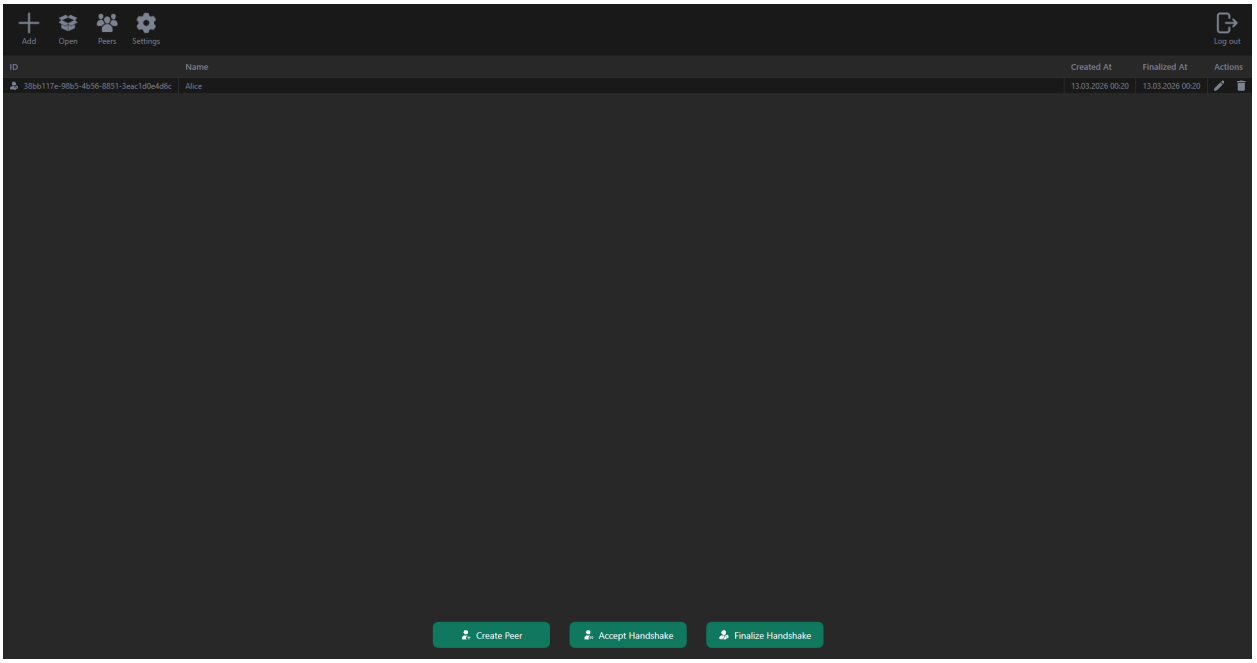


7.2.5. Löschen eines Peers

7.2.5.1. Klicken auf die Schaltfläche „Delete“ (rechts)

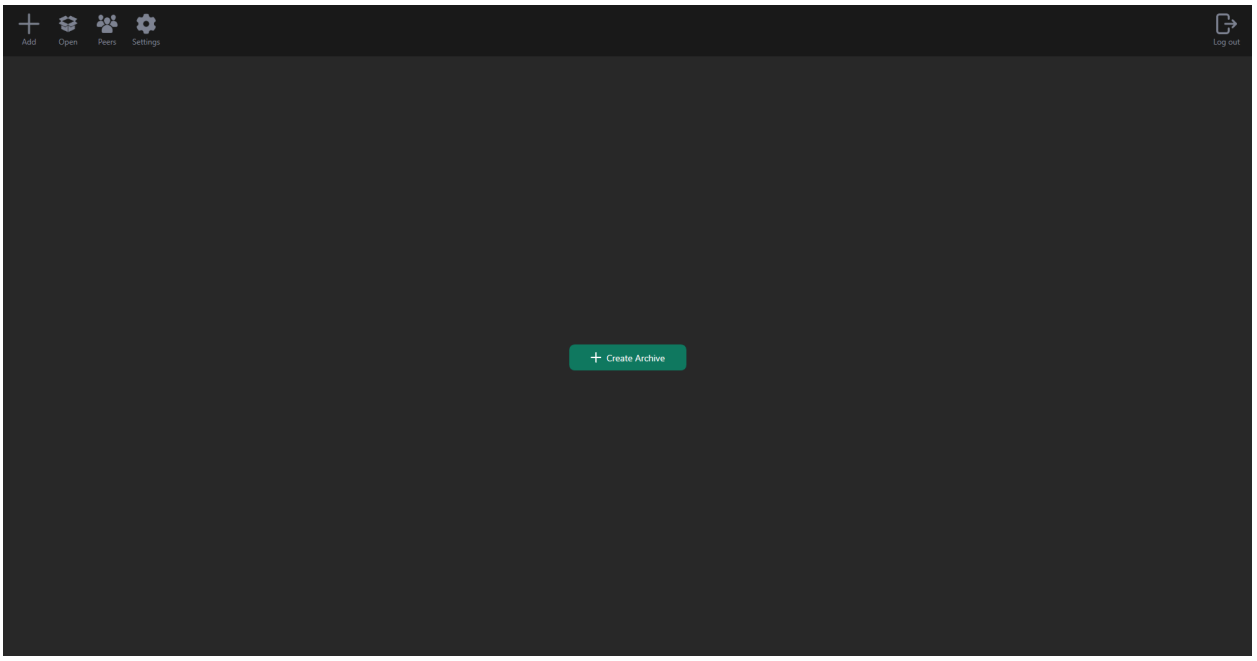


7.2.5.2. Löschvorgang des Peers „John“ abgeschlossen

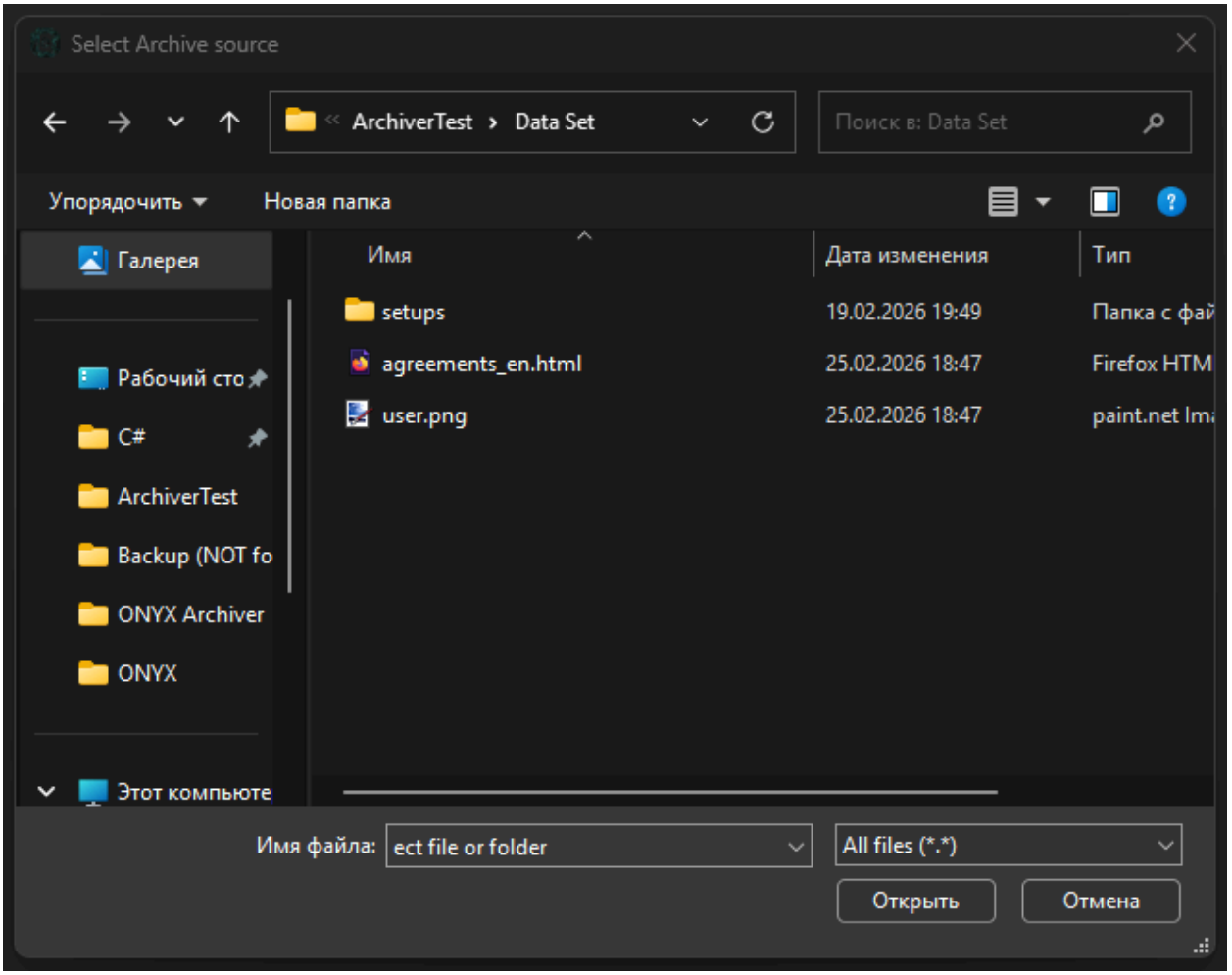


7.3. Erstellung des Archivs

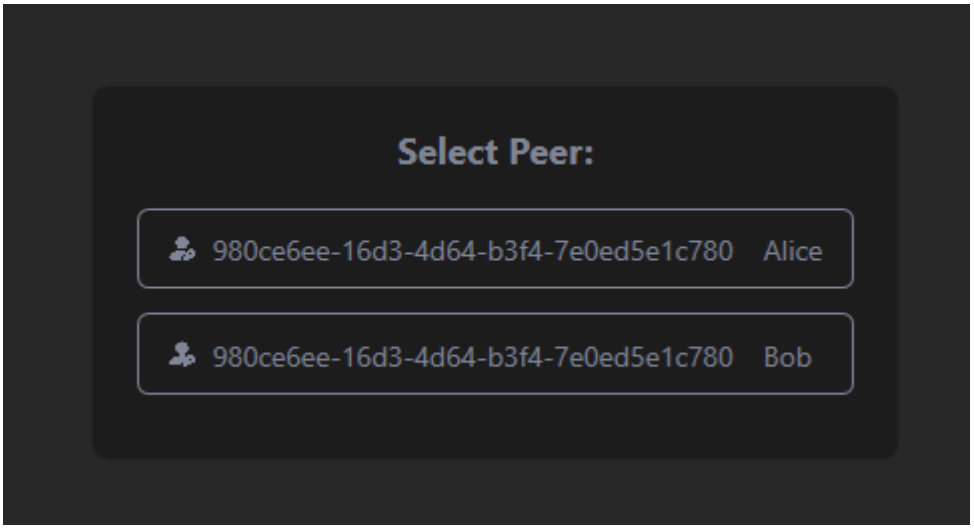
7.3.1. Klicken auf die Schaltfläche „Create Archive“



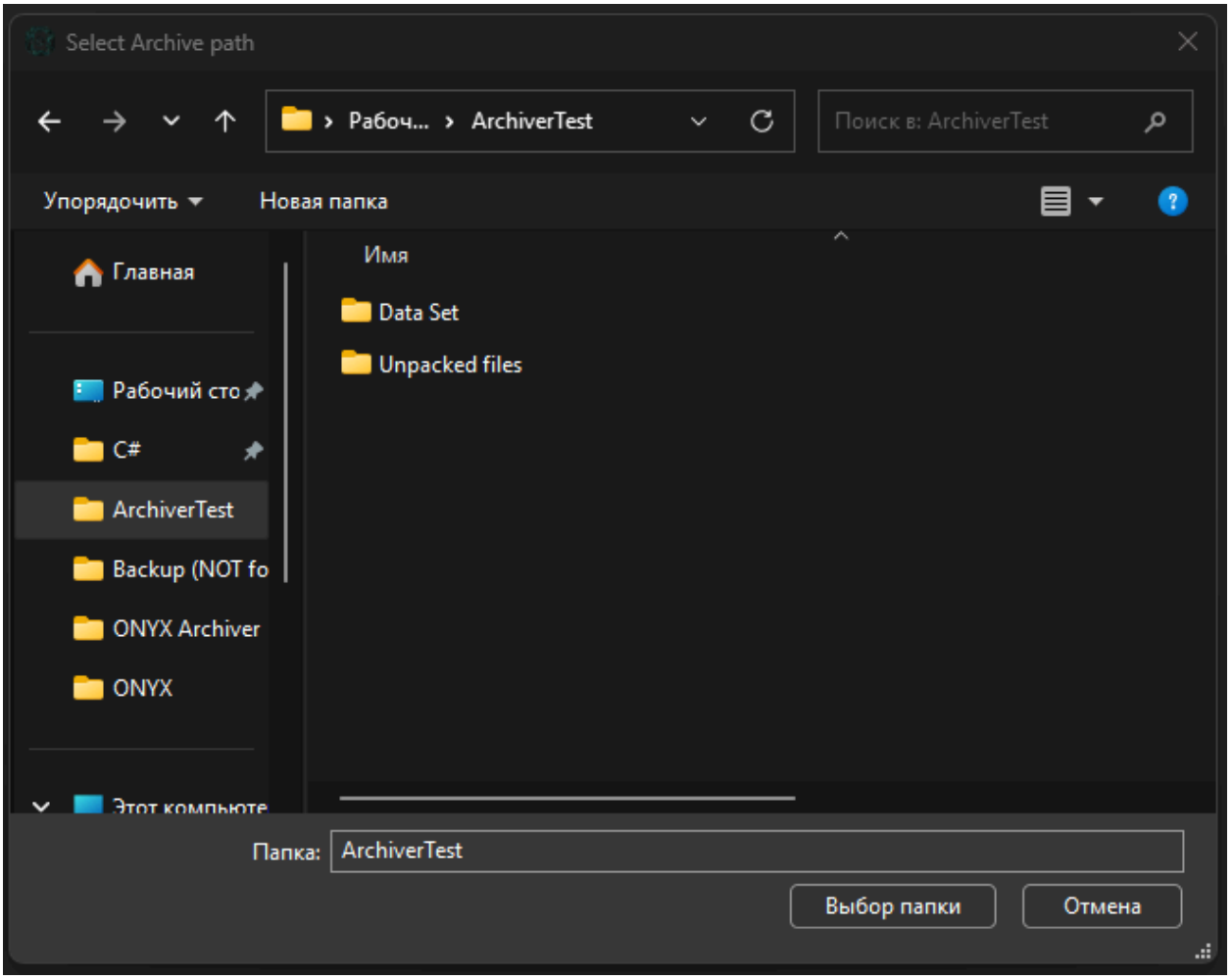
7.3.2. Auswahl des Quellverzeichnisses für die Archivierung



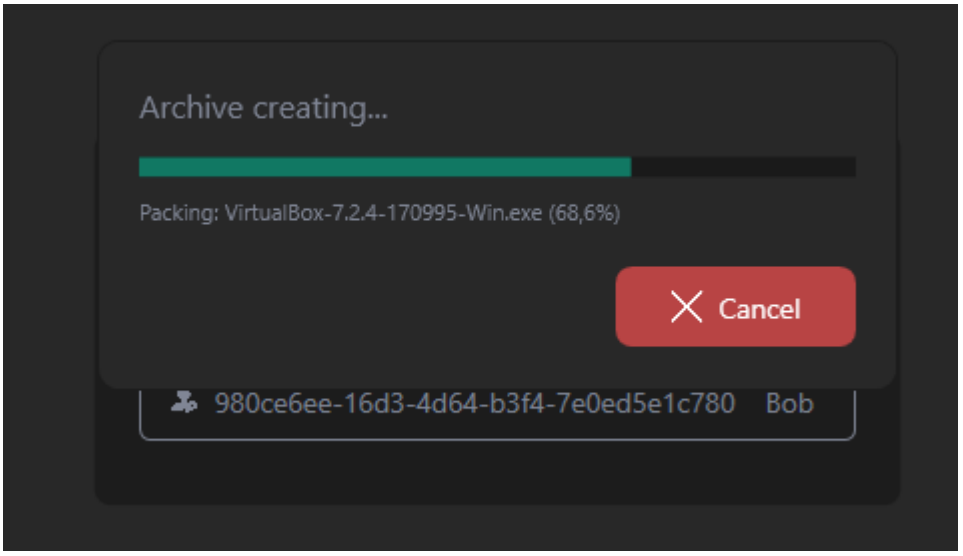
7.3.3. Auswahl des Zielpeers für die Archivverschlüsselung



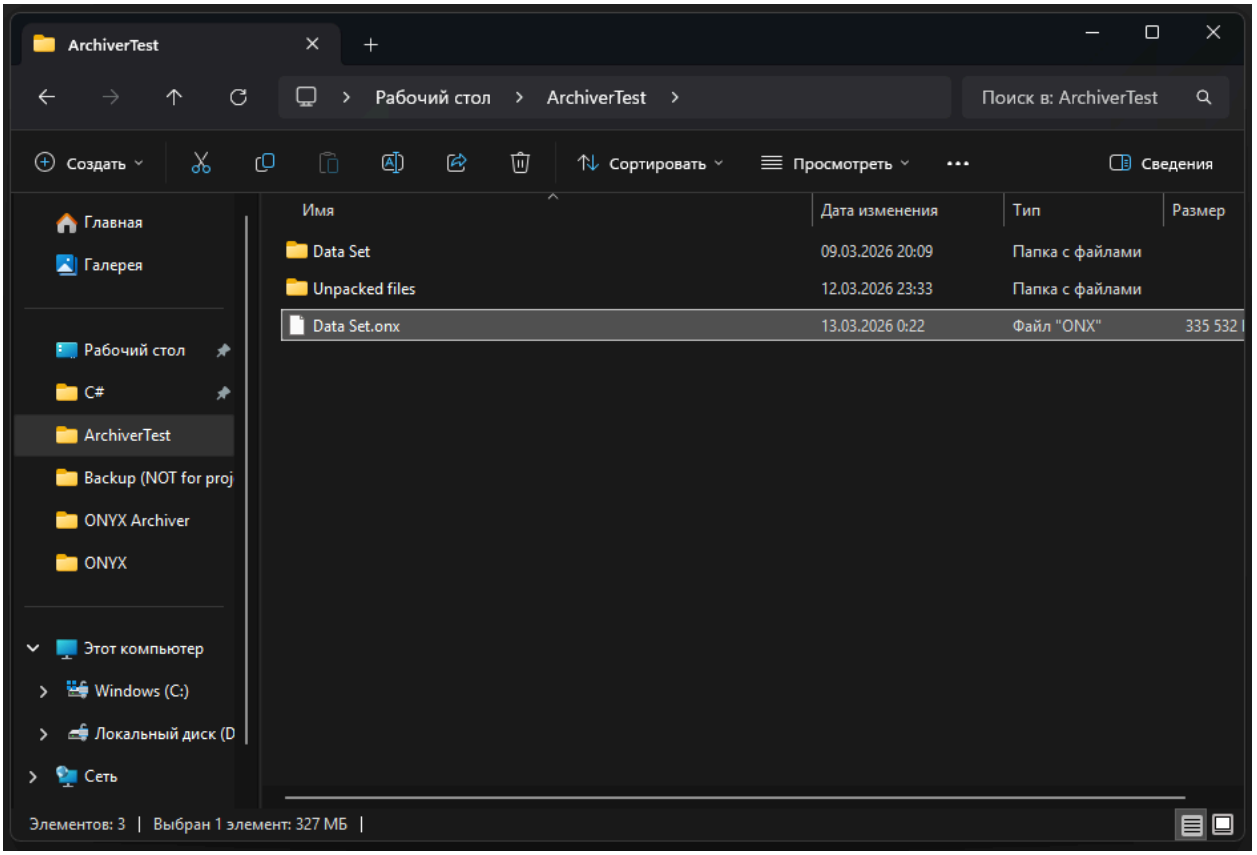
7.3.4. Zielverzeichnis für die Speicherung des Archivs festlegen



7.3.5. Fortschrittsfenster der Archiverstellung mit Prozessdetails

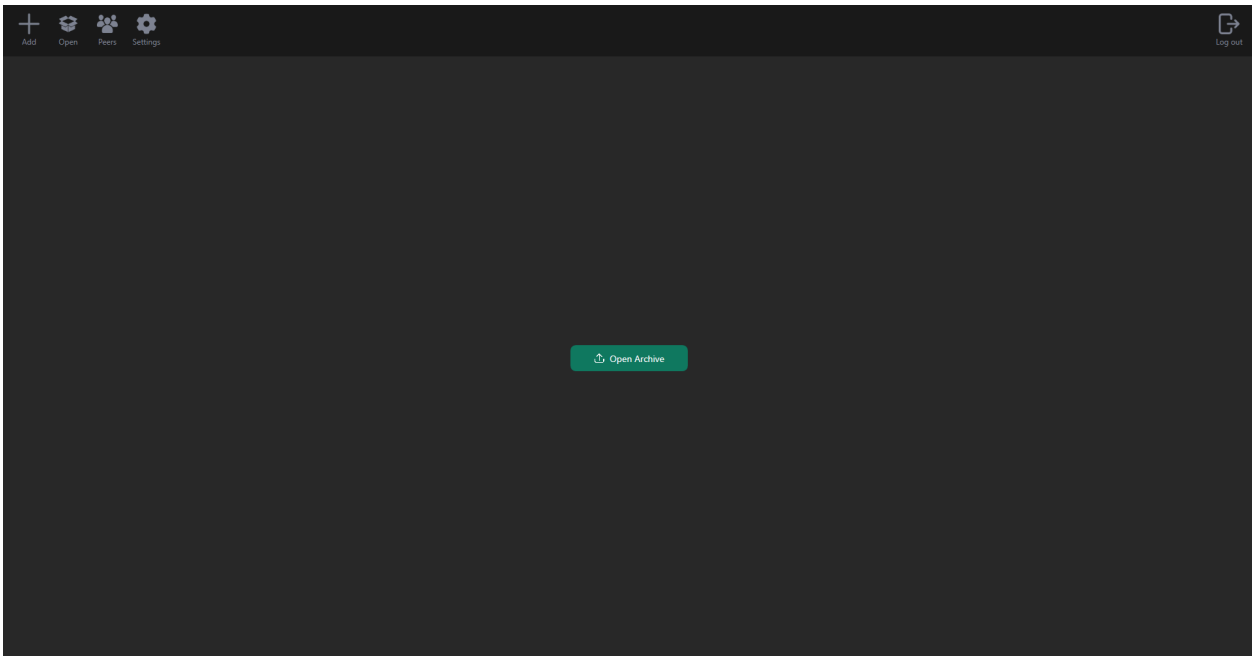


7.3.6. Archiv erfolgreich im Zielverzeichnis im Originalformat „.onx“ erstellt

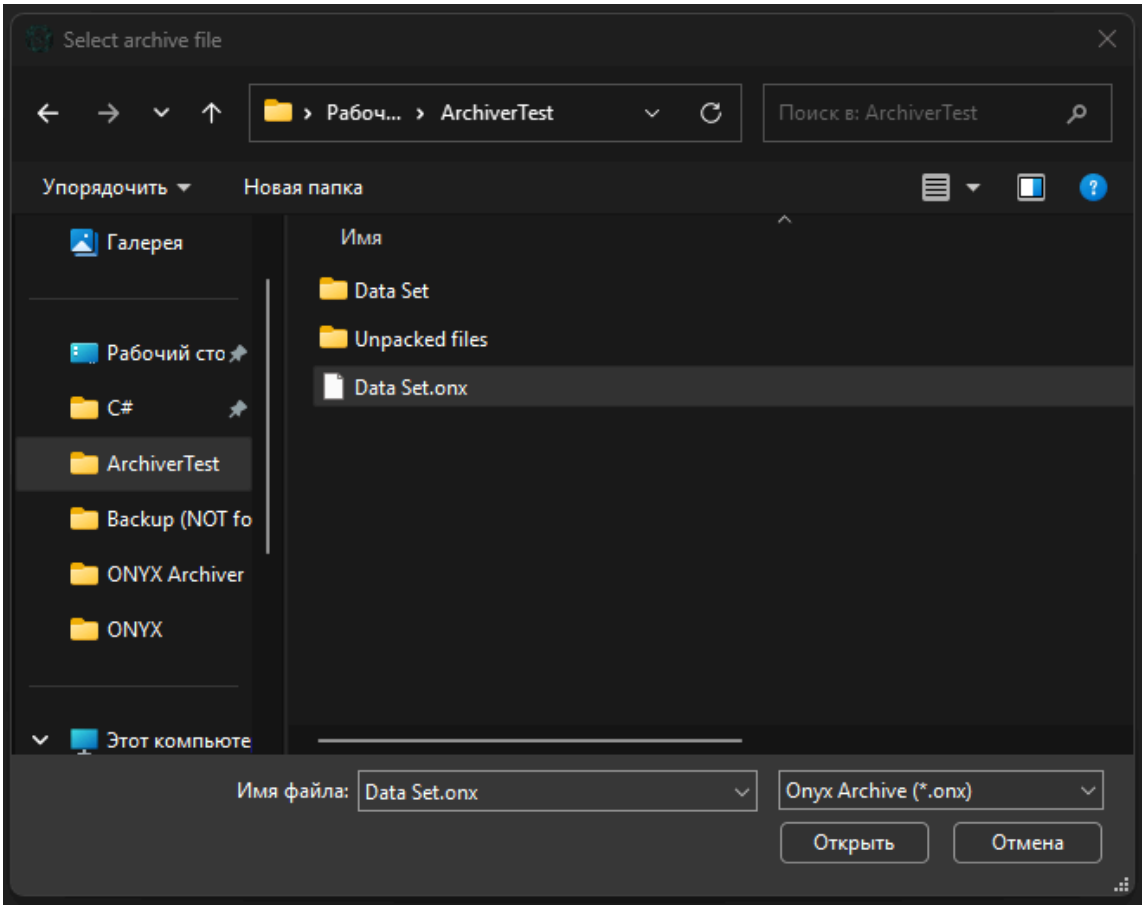


7.4. Öffnen des Archivs und Extraktion von Dateien

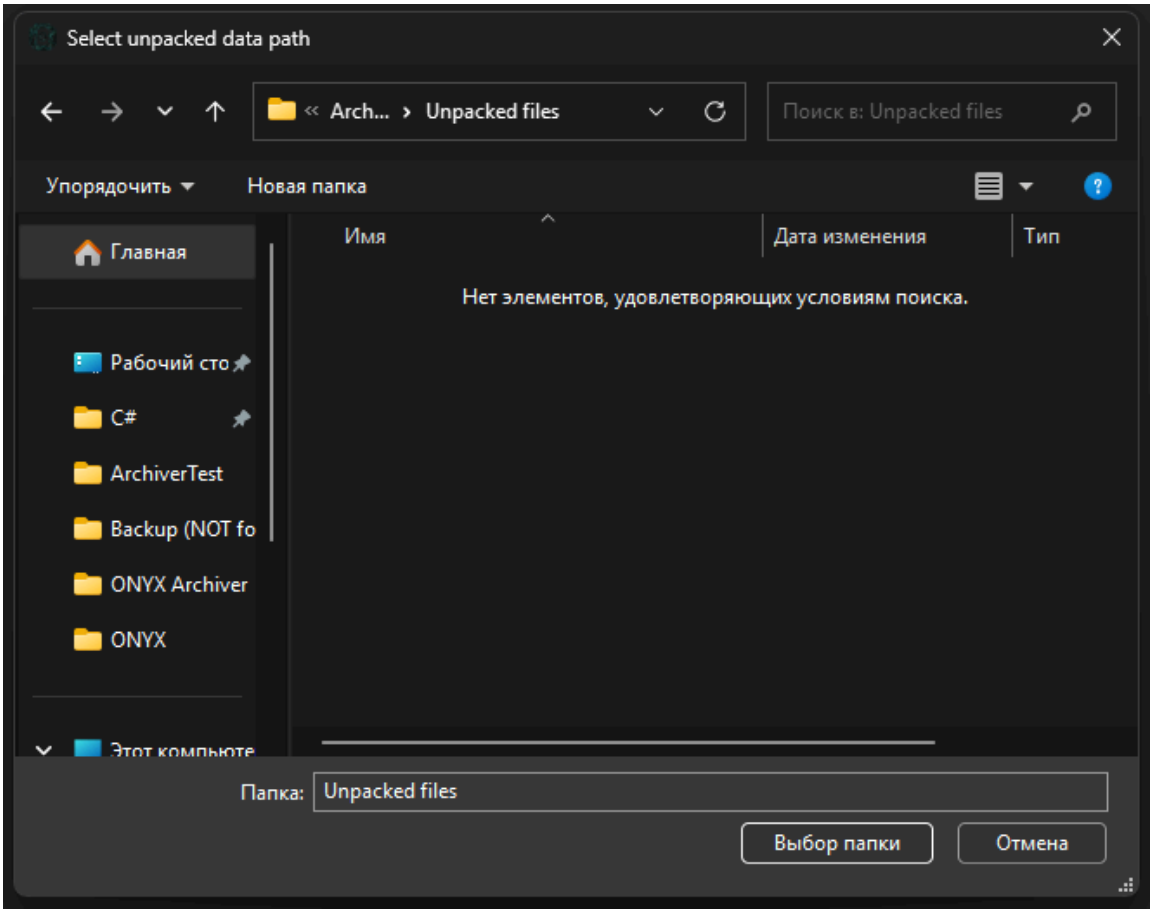
7.4.1. Klicken auf die Schaltfläche „Open Archive“



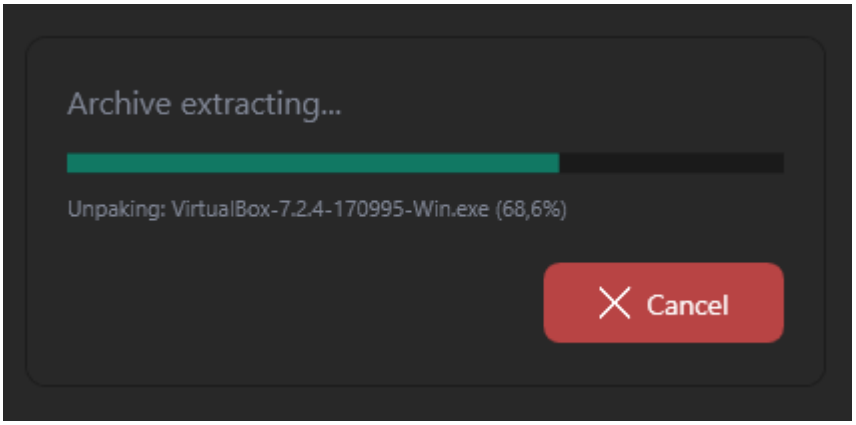
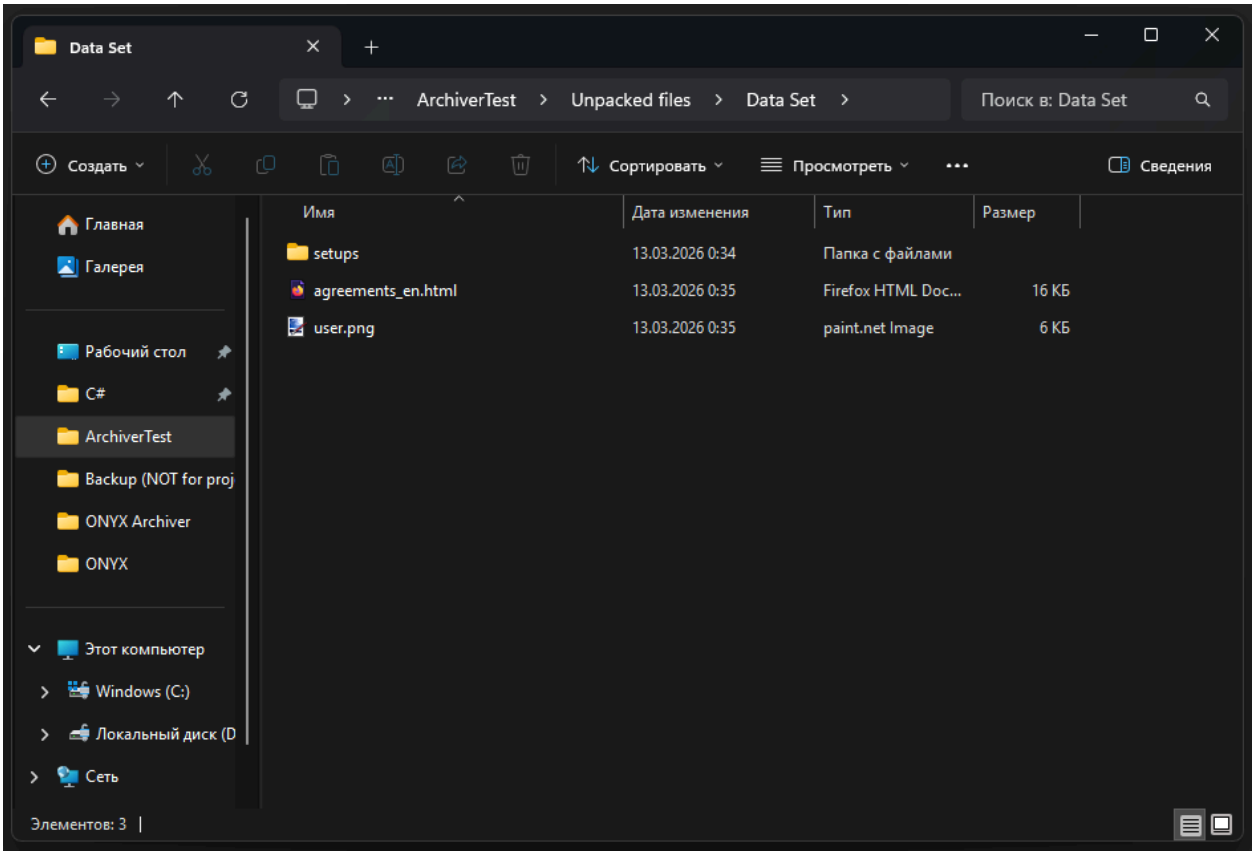
7.4.2. Auswahl der Archivdatei



7.4.3. Zielverzeichnis für die Extraktion der Dateien festlegen



7.4.4. Fortschrittsfenster der Dateiextraktion mit Prozessdetails



7.4.5. Erfolgreiche Extraktion des ursprünglichen Ordners und der Dateien